RUF — project roadmap

A poker hand probability calculator: from a Python engine to a public React site on Vercel, then back into R&D. Build inside out, and keep the engine free of any web code.

The path at a glance

#	Phase	What you end up with	You learn
0	Setup	Repo, Python venv, empty Vite app	Git, project layout
1	Engine	Python package that computes equity, with tests	Python, combinatorics, testing
2	API	FastAPI endpoint wrapping the engine	HTTP, JSON, schemas
3	React prototype	Card picker UI that shows the odds	React, TypeScript, state
4	Deploy	Public URL on Vercel	CI/CD, serverless limits
5	R&D	Climb the difficulty levels (separate page)	Bayes, performance, game theory

Ground rules

- **Engine first, web last.** The engine never imports FastAPI or React. That keeps it testable and portable (a phone app or a TypeScript port later).
- **Every phase ends with something that runs.** A terminal command, then a local API, then a page, then a URL.
- **Check numbers against known answers.** Example: AA vs KK all-in preflop is about 82% / 18%.
- **Fork points** mark places where another method is worth knowing. Take the simple path first and come back to them.

Folder layout

```
ruf/
├── engine/ruf_engine/   cards.py · evaluator.py · equity.py · assumptions.py
├── engine/tests/        pytest: known matchups, evaluator cross-checks
├── api/                   FastAPI app + Pydantic schemas
├── web/                   Vite + React + TypeScript
└── pyproject.toml / requirements.txt
```

Build phases

Phase 0 — Setup

Goal	A clean repo that every later phase drops into.
Tasks	• Create the folder layout above in your GitHub repo • Python virtual env: `python -m venv .venv` • Scaffold the frontend: `npm create vite@latest web -- --template react-ts` • Add a README with a one-line goal and how to run each part
Packages	git, Python 3.12+, Node.js
Done when	You can run an empty test suite and an empty React page.
Fork point	Package manager: **pip** is simplest; **uv** is faster and is worth trying later.

Phase 1 — Engine (Python)

Goal	Given your hand, the board, dead cards and opponent count, return win / tie / lose probabilities.
Tasks	• **Cards:** store each card as an integer 0–51 (rank × 4 + suit); add parse/format helpers for strings like "Ah" • **Evaluator, v1:** write a naive 7-card ranker yourself (try all 21 five-card subsets) • **Evaluator, v2:** swap in a library; keep v1 as a test oracle (both must agree on 10,000 random hands) • **Exact equity:** on the flop, 1,081 opponent hands × 990 runouts ≈ 1.07M matchups • **Monte Carlo equity:** for preflop / multiway; 10,000 samples ≈ ±1% at 95% confidence • **Assumptions:** dead cards and "opponent holds X" = remove or filter outcomes (conditioning) • **CLI:** `python -m ruf_engine AhKh "Qh Jh 2c"`
Packages	`phevaluator` or `treys`, `pytest`
Done when	Tests pass for known matchups, ties count as split pots, A-2-3-4-5 is a straight, and Monte Carlo agrees with exact counting to within its error bar.
Fork point	Evaluator choice (own vs treys vs phevaluator). Bitmask card representation. A precomputed table for the 169 starting hands.

Phase 2 API (FastAPI)

Goal	Expose the engine over HTTP without putting any math in the API layer.
Tasks	• One endpoint: `POST /api/equity` • Pydantic models for the request (hand, board, dead, opponents, samples) and the response • Validate input: no duplicate cards, board of 0/3/4/5 cards, opponents between 1 and 9 • Cap Monte Carlo samples on the server to stay inside serverless time limits • Try it locally at `localhost:8000/docs`
Packages	`fastapi`, `uvicorn` (local only), `pydantic`
Done when	The interactive docs page returns correct odds for a few hands you check by hand.
Fork point	Stateless API (no database) versus caching results. Streaming partial Monte Carlo results for a progress bar.

Phase 3 React prototype

Goal	A page where you pick cards and see the odds.
Tasks	• Components: **CardPicker**, **BoardInput**, **AssumptionsPanel**, **ResultsView** • State: selected cards live in one parent component; disable cards already in use • Call the API with `fetch`; show loading and error states • Show the result as a bar (win / tie / lose) plus the number and its ± error
Packages	`react`, `typescript`, `vite`; optional `tailwindcss`
Done when	You can enter any hand and board, and invalid input can't reach the API.
Fork point	Next.js instead of Vite (more features, more concepts). Running the engine in the browser later (see R&D Level 3).

Phase 4 Deploy (Vercel)

Goal	Public site with automatic deploys.
Tasks	• Push to GitHub, then import the repo into Vercel • Static React build served from the CDN; FastAPI runs as a Python function • Pin the Python version; keep dependencies small (Python functions aren't tree-shaken) • Every pull request gets a preview URL; merging to main ships to production • Add basic rate limiting before sharing the link widely (public, no sign-in)
Packages	Vercel account, GitHub
Done when	A friend can open the URL on their phone and get correct odds.
Fork point	Check the Python runtime's status (it was labeled Beta earlier in 2026). Custom domain. Analytics.

Phase 5 Back to R&D

Work through Levels 2–5 on the next page in order. Each level adds an engine feature first, then exposes it in the API and UI; keep the test suite growing so earlier levels never break.

R&D — levels of difficulty

Each level answers a harder question. Levels 1–3 are about probability and speed. Level 4 changes the question from how often do I win to what should I do.

Level	Question it answers	Key ideas	Milestone
1 Equity vs random	What are my odds if opponents hold random cards?	• Exact counting vs Monte Carlo • Ties, kickers, the wheel, dead cards • Standard error: √(p(1−p)/n)	Phases 1–4 done: live site with correct odds.
2 Ranges	What are my odds against what this player probably holds?	• A range = weights over the 1,326 starting hands (a prior) • Blockers: your cards make some of their hands impossible, so re-normalize • Evidence (actions) updates the range: Bayes' rule • Multiway: opponents aren't independent (same deck)	UI range picker (13×13 grid); the odds change with the range you choose.
3 Speed	Can I compute 6-way preflop with full ranges instantly?	• Precomputed tables (169 hand classes) • Suit symmetry: A♠K♠ ≡ A♥K♥ • Vectorize (numpy) or port to Rust / C • WebAssembly + Web Worker: run in the browser, no server	Benchmarks before and after; the browser engine matches the Python oracle.
4 Strategy	What should I do: check, bet, how much, how often to bluff?	• Game theory: Nash equilibrium in imperfect-information games • Counterfactual regret minimization (CFR) via self-play • Start with Kuhn poker (3 cards, ~100 lines), then Leduc	A CFR solver for Kuhn poker that converges to the known equilibrium.
5 Pluribus-style	How does a bot beat pros at 6-player no-limit?	• Abstraction: bucket similar hands (often by equity histograms, which reuse your Level 1–3 engine) and bet sizes • Monte Carlo CFR blueprint from self-play • Real-time depth-limited search during play	Reading project: reproduce a small abstracted hold'em solver; read Brown & Sandholm (2019).

The teal line marks the jump from probability (Levels 1–3) to decision-making (Levels 4–5).

Side track: mobile and Meta glasses

• Glasses apps run on the phone: Meta's Wearables Device Access Toolkit is an iOS (Swift) / Android (Kotlin) SDK. Ray-Ban Display also supports web apps.
• Your engine plugs in by calling the Vercel API, or by running a port on the device. A pure engine makes both possible.
• Reading cards from the camera is a computer-vision project bigger than the engine. Live odds during real play is real-time assistance, which casinos and online sites generally ban, so aim for study, review or training modes.